THE GREEDY METHOD IN KNAPSACK-TYPE PROBLEMS

Greedy reasoning reveals its strengths and limitations most clearly when applied to the knapsack problem, a classical optimization challenge that asks how to select items of different weights and benefits under a strict weight capacity. The slides open with a vivid scenario: a thief breaks into a museum filled with precious objects, each carrying a market value and a weight. His knapsack can only carry a limited load, and he must decide which items to take to maximize his total gain. This informal story motivates two closely related but fundamentally different variants of the knapsack problem: the **0–1 knapsack problem**, where items are indivisible, and the **fractional knapsack problem**, where items can be broken into parts.

In the **0–1 variant**, every item has a value viv_ivi and a weight wiw_iwi, and the capacity constraint is WWW. Because each item must be taken in its entirety or not at all, the search space is combinatorial. Greedy strategies, while appealing, often fail because taking the best-looking item at first glance can block better combinations later. In contrast, the **fractional knapsack problem** permits selecting any fraction of an item. Under this relaxation, the thief may treat objects like piles of gold dust rather than indivisible artifacts, allowing continuous allocation of weight capacity. This structural change transforms the problem into one perfectly suited for greedy optimization.

Though both versions exhibit optimal substructure, only the fractional version also satisfies the greedy-choice property. For the 0–1 case, if the optimal load includes an item jjj, then removing jjj leaves behind the optimal solution to capacity W−wjW - w_jW−wj drawn only from the remaining allowable items. For the fractional case, removing www units of item jjj leaves an optimal fractional allocation for capacity W−wW - wW−w, preserving problem structure throughout recursion.

Yet despite this shared optimal substructure, the 0–1 version lacks the essential guarantee that a locally best benefit-to-weight choice leads to a globally optimal outcome. This distinction appears immediately when comparing results: greedy selection succeeds perfectly for fractional knapsack and fails completely for 0–1 knapsack, underscoring how delicate the requirements for greedy optimality can be.

THE FRACTIONAL KNAPSACK ALGORITHM

The slides then present the fractional knapsack algorithm using clear pseudocode. Each item’s benefit-to-weight ratio bi/wib_i / w_ibi/wi is computed and assigned to viv_ivi. The greedy algorithm repeatedly chooses the item with the highest ratio and fills the knapsack with as much of that item as possible, updating the current used weight at each step. If an item cannot be taken fully, the algorithm includes exactly the fraction that fits and terminates.

The key insight is that the ratio vi=bi/wiv_i = b_i / w_ivi=bi/wi determines the order of selection, and that this ordering ensures optimality. The slides illustrate this with a concrete example, where the best benefit is obtained by taking full amounts of the top-rated items and then a fraction of the next one. In the example, the greedy fractional solution carries value

240, while the best 0–1 solution—restricted to whole items—achieves only 160. This gap directly reflects the presence or absence of the greedy-choice property.

Code of 0/1 Knapsack-

```
#include <bits/stdc++.h>

using namespace std;


int knapsackRec(int W, vector<int> &val, vector<int> &wt, int n) {


   if (n == 0 || W == 0)

      return 0;


   int pick = 0;


   if (wt[n - 1] <= W)

      pick = val[n - 1] + knapsackRec(W -

wt[n - 1], val, wt, n - 1);


   int notPick = knapsackRec(W, val, wt, n -

1);


   return max(pick, notPick);

}


int

knapsack(int W, vector<int> &val, vector<int> &wt) {

   int n = val.size();

   return knapsackRec(W, val, wt, n);
```

```
}

int main() {
   vector<int> val = {1, 2, 3};
   vector<int> wt = {4, 5, 1};
   int W = 4;

   cout << knapsack(W, val, wt) <<
endl;
   return 0;
}
```

EXERCISES ON KNAPSACK VARIANTS

To reinforce the conceptual distinction between the two knapsack versions, the slides include exercises. One exercise provides eight items with varying weights and profits and capacity W=16W = 16W=16, inviting full 0–1 knapsack reasoning. Another frames the knapsack problem as exam preparation: each question requires time and yields marks. With a fixed total study time TTT, the objective is to select questions that maximize total marks. This mapping to a knapsack structure reinforces how widely applicable the problem is across resource allocation settings.

THE GREEDY COINAGE PROBLEM

The chapter then transitions to a problem that appears to invite a greedy solution but, like the 0–1 knapsack problem, reveals its pitfalls in non-canonical environments. The coinage problem asks for the minimum number of coins needed to reach a target value VVV. The simple greedy strategy is to repeatedly select the largest coin denomination not exceeding the current remaining value. With familiar denominations such as {100,50,20,10,5,2,1}\{100, 50, 20, 10, 5, 2, 1\}{100,50,20,10,5,2,1}, this strategy performs perfectly; for V=93V = 93V=93, the greedy solution is {50,20,20,2,1}\{50, 20, 20, 2, 1\}{50,20,20,2,1}.

But the PDF emphasizes that greedy coin selection is **not always optimal**. With denominations {9,6,5,1}\{9, 6, 5, 1\}{9,6,5,1}, the greedy choice for V=11V = 11V=11 would be {9,1,1}\{9, 1, 1\}{9,1,1}, involving three coins, while the optimal answer {6,5}\{6, 5\}{6,5} uses only two. This example serves as a warning: greedy strategies succeed only when the structure of the problem supports them. In canonical currency systems, denomination relationships are designed so that greedy works; outside them, greedy collapses.

STRIP PACKING AND LEVEL-ORIENTED GREEDY METHODS

Greedy algorithms extend beyond numerical optimization and into geometric packing problems. The slides describe the two-dimensional strip packing problem, where items are axis-aligned rectangles and must be placed into a strip of fixed width and unbounded height without rotation and without overlap. The objective is to minimize total height.

A widely used approach for strip packing is the **level-oriented** strategy. Items are packed from left to right in horizontal rows or levels, with each level's height determined by the tallest rectangle placed on it. Once no new item can fit in the current level, a new level begins above it. This yields intuitive and structured packings that are easy to analyze.

To improve packing quality, items are commonly sorted in non-increasing height before placement. This leads to several variants:

- **First-Fit Decreasing Height (FFDH):** Each new item is placed in the first existing level where it fits; otherwise, a new level is opened.
- **Code for First fit –**

```
struct Rect { int w, h; };



int firstFit(vector<Rect>& r, int W) {

    vector<int> shelfWidth, shelfHeight;

    for(auto &x : r) {

        bool placed = false;

        for(int i=0;i<shelfWidth.size();i++) {

            if(shelfWidth[i] + x.w <= W) {

                shelfWidth[i] += x.w;

                shelfHeight[i] = max(shelfHeight[i], x.h);

                placed = true;

                break;

            }

        }

        if(!placed) {
```

```
            shelfWidth.push_back(x.w);
            shelfHeight.push_back(x.h);
        }
    }
    int total = 0;
    for(int h : shelfHeight) total += h;
    return total;
}
```

- **Next-Fit Decreasing Height (NFDH):** Items are placed only on the current level; if they do not fit, the entire level is closed and a new one started.
- **Code fo Next fit –**

```
struct Rect { int w, h; };


int nextFit(vector<Rect>& r, int W) {
    int usedWidth = 0, shelfHeight = 0, totalHeight = 0;
    for(auto &x : r) {
        if(usedWidth + x.w > W) {
            totalHeight += shelfHeight;
            usedWidth = 0;
            shelfHeight = 0;
        }
        usedWidth += x.w;
        shelfHeight = max(shelfHeight, x.h);
    }
    return totalHeight + shelfHeight;
}
```

- **Best-Fit Decreasing Height (BFDH):** The item is placed on the level that leaves the least unused horizontal space among levels it fits in.
- **Code for First fit –**

```
struct Rect { int w, h; };


int bestFit(vector<Rect>& r, int W) {
    vector<int> shelfWidth, shelfHeight;


    for(auto &x : r) {
        int best = -1, minSpace = 1e9;


        for(int i=0;i<shelfWidth.size();i++) {
            int space = W - shelfWidth[i];
            if(x.w <= space && space < minSpace) {
                minSpace = space;
                best = i;
            }
        }


        if(best == -1) {
            shelfWidth.push_back(x.w);
            shelfHeight.push_back(x.h);
        } else {
            shelfWidth[best] += x.w;
            shelfHeight[best] = max(shelfHeight[best], x.h);
        }
```

```
        }

        int total = 0;

        for(int h : shelfHeight) total += h;

        return total;

    }
```

The PDF also mentions the **Bottom-Left (BL)** method, a non-level strategy that places each item as low and as far left as possible without overlap. BL can achieve tighter packing but at higher computational cost, often quadratic in the number of items.

THE BIN PACKING PROBLEM

The bin packing problem represents another fundamental packing scenario. Given bins of uniform capacity and a set of items each with its own weight, the goal is to minimize the number of bins used. The slides provide an example where items {4,8,1,4,2,1}\{4, 8, 1, 4, 2, 1\}{4,8,1,4,2,1} must be packed into bins of capacity 10. One solution uses only two bins: {4,4,1,1}\{4, 4, 1, 1\}{4,4,1,1} and {8,2}\{8, 2\}{8,2}.

A simple greedy heuristic sorts items by decreasing weight and inserts each into the most recently used bin if it fits; otherwise, a new bin is opened. While not always optimal, this approach is effective in many practical settings and illustrates how greedy strategies can provide strong approximate solutions even when optimality is not guaranteed.

GREEDY TASK SCHEDULING WITH DEADLINES AND PENALTIES

Another significant application of greedy methods appears in scheduling tasks with deadlines. Each task takes exactly one unit of time and incurs a penalty if completed after its deadline. The goal is to schedule tasks to **minimize the total penalty**, ensuring high-penalty tasks are prioritized.

A crucial insight from the slides is that schedules can always be converted into canonical form, with early tasks arranged first and in order of increasing deadlines. A set of tasks is independent if they can all be scheduled without lateness. Several equivalent properties characterize independent sets:
they must satisfy that, for every ttt, at most ttt tasks have deadlines ≤t\leq t≤t, and when sorted by deadline, no task is late.

The greedy algorithm for selecting early tasks sorts tasks in **non-increasing order of penalty** and attempts to schedule each as late as possible before its deadline. If a suitable slot exists, the task is placed; otherwise, it is excluded, contributing to total penalty. This method ensures

that losses from unavoidable lateness are minimized. The slides provide a detailed numerical example showing the iterative placement of tasks and computation of total penalty.

SHORTEST JOB FIRST (SJF) SCHEDULING

The slides then introduce SJF, a greedy scheduling policy in operating systems. SJF selects the job with the smallest burst time to run next, minimizing average waiting time. In the non-preemptive form, once a job begins, it runs to completion. The preemptive version—Shortest Remaining Time First—always selects the job with the smallest remaining time at any moment, preempting longer jobs that arrive later.

Includes detailed scheduling tables and Gantt charts showing how waiting time and turnaround time are calculated for both variants, illustrating how SJF optimizes time-related performance metrics while potentially causing starvation if short jobs continue to arrive.

Code for Shortest Job First-

```
// Sort processes by arrival time (helper)

void sortByArrival(vector<int> &id, vector<int> &at, vector<int> &bt) {

    int n = id.size();

    vector<int> o(n);

    iota(o.begin(), o.end(), 0);



    sort(o.begin(), o.end(), [&](int a, int b){ return at[a] < at[b]; });



    vector<int> ID(n), AT(n), BT(n);

    for (int i = 0; i < n; i++) {

        ID[i] = id[o[i]];

        AT[i] = at[o[i]];

        BT[i] = bt[o[i]];

    }
```

```
    id = ID; at = AT; bt = BT;
}

// SJF Scheduling (Greedy)
void SJF(vector<int> &id, vector<int> &at, vector<int> &bt) {
    int n = id.size(), time = 0, completed = 0;
    vector<bool> done(n, false);

    while (completed < n) {
        int idx = -1, mn = INT_MAX;

        // pick shortest available job
        for (int i = 0; i < n; i++)
            if (!done[i] && at[i] <= time && bt[i] < mn)
                mn = bt[i], idx = i;

        // no job available → move time
        if (idx == -1) { time++; continue; }

        time += bt[idx];
        done[idx] = true;
        completed++;

        cout << "P" << id[idx] << " finishes at " << time << "\n";
    }
```

}

GRAPH COLORING AND ITS GREEDY APPROXIMATION

Graph coloring assigns colors to vertices such that adjacent vertices have different colors. The minimum number of colors needed is the **chromatic number**. The greedy coloring algorithm processes vertices in some order and assigns each the smallest available color not used by its neighbors.

Although greedy coloring is simple and often effective, its performance depends heavily on vertex order. The slides present two graphs where processing the same vertices in different orders results in the use of three colors or four colors. Nonetheless, a theoretical guarantee ensures that the greedy algorithm never uses more than Δ+1\Delta + 1Δ+1 colors, where Δ\DeltaΔ is the maximum degree.

This section emphasizes that greedy coloring is not optimal but offers a practical approximation with strong upper bounds.

GREEDY CODING METHODS: HUFFMAN AND SHANNON–FANO

The final topic in the PDF examines greedy methods in text compression. Both Huffman coding and Shannon–Fano coding assign shorter codes to more frequent characters and longer codes to less frequent ones, reducing overall encoded length.

Huffman coding constructs an optimal prefix-free code by repeatedly combining the two lowest-frequency nodes into a new internal node. The process continues until the entire-frequency hierarchy becomes a binary tree whose structure directly determines the code lengths. The PDF includes a complete worked example, showing tree construction, final codes for characters a,b,c,d,e,fa, b, c, d, e, fa,b,c,d,e,f, and analysis of encoding time complexity, which is bounded by O(nlog⁡n)O(n \log n)O(nlogn) due to repeated heap operations.

Shannon–Fano coding, shown next, proceeds by sorting characters by decreasing probability and recursively partitioning the list into two equal-weight halves, assigning 0 to one side and 1 to the other. Although not always optimal like Huffman coding, it remains an important historical greedy approach. The PDF provides a full Shannon–Fano tree, final codes, and a calculation comparing compression ratios to ASCII encoding.